Name : T Bhanu Teja

Reg no: 192472259

UNIT II

FUNCTIONS AND CLASSES & OBJECTS IN C++

C++ Programming – Questions with Answers

Syllabus scope: main function, function prototypes, call by reference, return by reference, inline functions, default arguments, function overloading, classes and member functions, arrays within a class, static members, arrays of objects, and friend functions.

Assessment Answer Format

Section A – Debugging: Each answer identifies the defective line and error category, explains the root cause, provides complete corrected C++ code, and includes a dry run with the predicted output.

Section B – Pseudocode: Each answer includes a brief design overview, structured pseudocode using the prescribed class and method syntax, and a trace table using the sample input.

SECTION A: DEBUGGING QUESTIONS

Each debugging question is presented with its topic, problem statement, original buggy code, a brief answer introduction, corrected C++ code, dry run, and exact output.

Question 1: Dangling Reference in Return-by-Reference

Topic: Return by Reference, Scope, Activation Records.

Problem Statement: A utility returns the scaled maximum of two integers by reference. Returning its local result causes undefined behavior, while returning an element of an array passed by reference is safe.

Buggy Code:

```
#include <iostream>

// Line 4: Utility to scale and return the larger value
```

```
int& getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    int result = maxVal * scaleFactor;
    return result; // Warning / Undefined Behavior!
}

// Line 11: Array element modifier
int& getArrayElement(int arr[], int index) {
    return arr[index];
}

int main() {
    int x = 10, y = 20;

    // Line 19: Receiving the reference
    int &danglingRef = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << danglingRef << std::endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999; // Lvalue assignment
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    // Line 27: Reading reference after subsequent stack calls
    std::cout << "Dangling Ref Value: " << danglingRef << std::endl;
    return 0;
}
```

ANSWER

The function getMaximumScaled returns a reference to a local variable. That variable ceases to exist when the function returns. The array function returns a reference to an element that remains alive in the caller-owned array.

1. Line number and error classification / root cause: Line 7 returns the local variable result by reference, creating a dangling reference and undefined behavior. Line 19 binds danglingRef to that invalid reference. getArrayElement is safe when the supplied array remains alive and the index is valid.

2. Root cause and correction: Return the calculated integer by value. Keep getArrayElement returning int& because it refers to an existing array element.

3. Corrected C++ code:

```
#include <iostream>

int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor;
}
```

```
int& getArrayElement(int arr[], int index) {
   return arr[index];
}

int main() {
   int x = 10, y = 20;

   int scaledMax = getMaximumScaled(x, y, 2);
   std::cout << "Scaled Max: " << scaledMax << std::endl;

   int data[3] = {100, 200, 300};
   getArrayElement(data, 1) = 999;
   std::cout << "Updated Array[1]: " << data[1] << std::endl;

   std::cout << "Scaled Max Value: " << scaledMax << std::endl;
   return 0;
}
```

4. Dry run: max(10,20) = 20; scaledMax = 20 × 2 = 40. The array element at index 1 changes from 200 to 999. scaledMax remains a valid independent integer.

Step	Operation	Result
1	getMaximumScaled(10, 20, 2)	Returns 40 by value
2	data = {100, 200, 300}	Array initialized
3	getArrayElement(data, 1) = 999	data[1] becomes 999
4	Print values	40 and 999

4. Exact output:

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Max Value: 40
```

Question 2: Static Data Member Linkage & Member Function Scoping

Topic: Static Data Members, Static Member Functions, Scope Resolution.

Problem Statement: BankAccount tracks total branch reserves using a static variable. The static audit function incorrectly attempts to access the instance variable accHolder, and the static data member lacks its required definition.

Buggy Code:

```
#include <iostream>
#include <string>
```

```
class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves; // Line 8: Static data member

public:
    BankAccount(std::string name, double initialDeposit) {
        accHolder = name;
        balance = initialDeposit;
        totalBankReserves += initialDeposit;
    }

    // Line 17: Static member function
    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $" << totalBankReserves << std::endl;
        std::cout << "Account Holder: " << accHolder << std::endl; // Error!
    }
};

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    BankAccount::displayAudit();
    return 0;
}
```

ANSWER

1. Line number and error classification / root cause: The declaration static double totalBankReserves; inside the class declares the member but, in this pre-C++17 style, does not provide its storage definition. Define it once outside the class. Also, displayAudit() is static and has no this pointer, so it cannot access accHolder directly.

2. Corrected C++ code:

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;

public:
    BankAccount(std::string name, double initialDeposit) {
```

```
        accHolder = name;
        balance = initialDeposit;
        totalBankReserves += initialDeposit;
    }

    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder << std::endl;
        std::cout << "Balance: $" << balance << std::endl;
    }

    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $"
                  << totalBankReserves << std::endl;
    }
};

double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();
    BankAccount::displayAudit();
    return 0;
}
```

3. Dry run: acc1 adds $5,000 to the static total. acc2 adds $3,000. The aggregate reserve is $8,000. Each object method displays its own account details.

Step	**Action**	**totalBankReserves**
1	Initialize static member	0
2	Create Alice account with 5000	5000
3	Create Bob account with 3000	8000
4	Call BankAccount::displayAudit()	8000

4. Exact output:

```
Account Holder: Alice
Balance: $5000
Account Holder: Bob
Balance: $3000
Total Reserves in Vault: $8000
```

Question 3: Function Overload Ambiguity via Default Arguments

Topic: Function Overloading, Default Arguments.

Problem Statement: A geometry library overloads computeVolume for cuboids, cubes, and cylinders. The one-argument call is ambiguous because both the cuboid overload with default arguments and the cube overload are viable.

Buggy Code:

```
#include <iostream>

// Overload 1: Cuboid volume with default dimensions
double computeVolume(double length, double width = 1.0, double height = 1.0) {
    return length * width * height;
}

// Overload 2: Dedicated function for cube volume
double computeVolume(double side) {
    return side * side * side;
}

// Overload 3: Cylinder volume calculation
double computeVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
}

int main() {
    // Line 19: Call with 1 argument fails compilation
    std::cout << "Volume: " << computeVolume(5.0) << std::endl;

    // Line 22: Call with 2 arguments
    std::cout << "Cylinder: " << computeVolume(3.0, 7.0) << std::endl;
    return 0;
}
```

ANSWER

1. Line number and error classification / root cause: computeVolume(5.0) can match the cuboid overload by supplying default width and height, or match the dedicated one-argument cube overload. Both are exact matches, so the call is ambiguous. The two-argument call also matches both the cuboid overload (height defaults) and cylinder overload, so it is ambiguous as well.

2. Root cause and correction: Use distinct function names to make the intended shape explicit and eliminate overload ambiguity.

3. Corrected C++ code:

```
#include <iostream>

double cuboidVolume(double length, double width = 1.0,
                    double height = 1.0) {
   return length * width * height;
}

double cubeVolume(double side) {
   return side * side * side;
}

double cylinderVolume(double radius, double height) {
   return 3.14159 * radius * radius * height;
}

int main() {
   std::cout << "Cube: " << cubeVolume(5.0) << std::endl;
   std::cout << "Cuboid: " << cuboidVolume(5.0, 2.0, 3.0) << std::endl;
   std::cout << "Cylinder: " << cylinderVolume(3.0, 7.0) << std::endl;
   return 0;
}
```

4. Dry run: Cube = $5^3 = 125$. Cuboid = $5 \times 2 \times 3 = 30$. Cylinder = $3.14159 \times 3 \times 3 \times 7 \approx 197.92017$.

4. Exact output:

```
Cube: 125
Cuboid: 30
Cylinder: 197.92
```

Question 4: Array of Objects Missing Default Constructor

Topic: Arrays of Objects, Constructors, Class Specification.

Problem Statement: Student defines only a parameterized constructor, then declares Student batch[3]. Each array element needs to be default-constructed, but no accessible default constructor exists.

Buggy Code:

```
#include <iostream>
#include <string>

class Student {
private:
   int rollNumber;
   std::string name;

public:
```

```
    // Parameterized constructor defined by programmer
    Student(int r, std::string n) {
        rollNumber = r;
        name = n;
    }

    void display() const {
        std::cout << "Roll: " << rollNumber << ", Name: " << name << std::endl;
    }
};

int main() {
    // Line 20: Array of 3 Student objects
    Student batch[3]; // Compilation Error!

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

ANSWER

1. Line number and error classification / root cause: Compilation error at Student batch[3]. When a programmer declares a parameterized constructor, the compiler does not automatically generate a default constructor. The array declaration requires a zero-argument constructor for each element.

2. Corrected C++ code (Technique A – explicit default constructor):

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() {
        rollNumber = 0;
        name = "";
    }

    Student(int r, std::string n) {
```

```
        rollNumber = r;
        name = n;
    }

    void display() const {
        std::cout << "Roll: " << rollNumber
                  << ", Name: " << name << std::endl;
    }
};

int main() {
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

3. Dry run: The array creates three objects using the default constructor. The assignments then replace each object's values with the supplied roll number and name.

Index	Assigned object	Displayed result
0	Student(101, "Aarav")	Roll: 101, Name: Aarav
1	Student(102, "Bhavna")	Roll: 102, Name: Bhavna
2	Student(103, "Chirag")	Roll: 103, Name: Chirag

4. Exact output:

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration & Scope Resolution Flaws

Topic: Friend Functions, Forward Declarations, Member vs. Non-Member.

Problem Statement: Engine and TelemetryAuditor use a non-member friend function to inspect private data. The code lacks the forward declaration of TelemetryAuditor, accesses alertThreshold without an object, and invokes the friend function as a member.

Buggy Code:

```
#include <iostream>

// Line 4: Incomplete forward declarations
class Engine;

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    // Declaring friend function
    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}

    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

// Line 27: Friend function implementation
void inspectDiagnostics(Engine e, TelemetryAuditor a) {
    // Line 29: Unqualified member access error
    if (e.temperature > alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at " << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    // Line 41: Invoking friend function as a member function
    v8.inspectDiagnostics(v8, audit);
    return 0;
}
```

ANSWER

1. Line number and error classification / root cause: TelemetryAuditor must be forward-declared before it is used in Engine's friend declaration. A friend function is not a member and has no this pointer. It must access private members through the explicitly passed objects, such as a.alertThreshold. It is called as inspectDiagnostics(v8, audit), not v8.inspectDiagnostics(...).

2. Corrected C++ code:

```
#include <iostream>

class Engine;
class TelemetryAuditor;

void inspectDiagnostics(const Engine& e,
                        const TelemetryAuditor& a);

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};

void inspectDiagnostics(const Engine& e,
                        const TelemetryAuditor& a) {
    if (e.temperature > a.alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at "
                  << e.rpm << " RPM." << std::endl;
    }
}

int main() {
```

```
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);
    return 0;
}
```

3. Dry run: Engine temperature is 105 and the auditor threshold is 100. Since 105 > 100, the warning branch executes.

4. Exact output:

```
Warning: Engine overheat!
```

SECTION B: PSEUDOCODE WRITING QUESTIONS

Each pseudocode question is presented with its topic, scenario, student tasks, structured pseudocode answer, and sample-input trace table.

Question 1: Corporate Vault Management with Static Data & Functions

Topic: Static Data Members, Static Member Functions, Encapsulation.

Scenario: Design a banking branch reserve system. Each BranchOffice stores branchID, branchName and currentBalance. Static attributes track totalVaultReserves and totalBranchesActive. Constructors add opening deposits and increment branch count. Deposit updates branch and corporate balances. Withdraw succeeds only when sufficient branch funds exist. Static methods display the consolidated summary and calculate totalVaultReserves / targetReserve.

Student Tasks: Define BranchOffice and Main. Trace two branches, including Branch1 depositing $150,000 and Branch2 withdrawing $50,000.

ANSWER

1. Design overview: Instance data belongs to each branch; static data is shared by every BranchOffice object. Instance transactions update both the branch balance and the shared reserve.

2. Structured pseudocode:

```
CLASS BranchOffice
PRIVATE:
  branchID : STRING
  branchName : STRING
  currentBalance : REAL
  STATIC totalVaultReserves : REAL <- 0.0
  STATIC totalBranchesActive : INTEGER <- 0

PUBLIC:
  CONSTRUCTOR(id : STRING, name : STRING, openingDeposit : REAL)
    branchID <- id
    branchName <- name
    currentBalance <- openingDeposit
    totalVaultReserves <- totalVaultReserves + openingDeposit
    totalBranchesActive <- totalBranchesActive + 1
  ENDCONSTRUCTOR

  METHOD Deposit(amount : REAL)
    IF amount > 0 THEN
      currentBalance <- currentBalance + amount
      totalVaultReserves <- totalVaultReserves + amount
    ELSE
      DISPLAY "Invalid deposit"
    ENDIF
  ENDMETHOD
```

```
  METHOD Withdraw(amount : REAL) RETURNS BOOLEAN
    IF amount > 0 AND amount <= currentBalance THEN
      currentBalance <- currentBalance - amount
      totalVaultReserves <- totalVaultReserves - amount
      RETURN TRUE
    ELSE
      RETURN FALSE
    ENDIF
  ENDMETHOD

  STATIC METHOD GetVaultSummary()
    DISPLAY "Active branches: ", totalBranchesActive
    DISPLAY "Total reserves: ", totalVaultReserves
  ENDMETHOD

  STATIC METHOD GetGlobalLiquidityRatio(targetReserve : REAL) RETURNS REAL
    IF targetReserve <= 0 THEN
      RETURN 0
    ENDIF
    RETURN totalVaultReserves / targetReserve
  ENDMETHOD
ENDCLASS

BEGIN
  DECLARE Branch1 : BranchOffice
  DECLARE Branch2 : BranchOffice
  Branch1 <- CONSTRUCT("B01", "North", 200000)
  Branch2 <- CONSTRUCT("B02", "South", 100000)
  Branch1.Deposit(150000)
  Branch2.Withdraw(50000)
  BranchOffice.GetVaultSummary()
  DISPLAY "Liquidity ratio: ",
        BranchOffice.GetGlobalLiquidityRatio(500000)
END
```

3. Dry-run trace: Assume opening deposits of $200,000 for Branch1 and $100,000 for Branch2, since the question does not specify opening balances. Branch1 deposits $150,000; Branch2 withdraws $50,000.

Step	**Transaction**	**Branch 1**	**Branch 2**	**Static total reserves**
Initial	Create both branches	$200,000	$100,000	$300,000
1	Branch1 deposits $150,000	$350,000	$100,000	$450,000
2	Branch2 withdraws $50,000	$350,000	$50,000	$400,000

Final active branches: 2. Final corporate reserves: $400,000. Liquidity ratio against $500,000 target = 0.8.

Question 2: Matrix Mathematical Engine with Arrays in Class & Friend Dot-Product

Topic: Arrays within a Class, Scope Resolution ::, Friend Functions.

Scenario: Design Matrix2D to encapsulate a 3 × 3 REAL array. Initialize it to zeros; set and get elements with bounds checks; display it. Define member methods outside the class using Matrix2D::MethodName(). A friend function ComputeCrossTrace(A, B) directly computes the dot product of the two primary diagonals.

Student Tasks: Write class declarations, outside method implementations and friend function. Trace diagonal values A = [2,4,6] and B = [1,3,5].

ANSWER

1. Design overview: The matrix grid is private. Public methods validate indices before accessing elements. The friend function has permission to access both objects' private grids directly.

2. Structured pseudocode:

```
CLASS Matrix2D
PRIVATE:
  grid : ARRAY[3][3] OF REAL
PUBLIC:
  CONSTRUCTOR()
  METHOD Initialize()
  METHOD SetElement(row : INTEGER, col : INTEGER, value : REAL)
  METHOD GetElement(row : INTEGER, col : INTEGER) RETURNS REAL
  METHOD Display()
  FRIEND FUNCTION ComputeCrossTrace(A : Matrix2D, B : Matrix2D) RETURNS REAL
ENDCLASS

METHOD Matrix2D::Initialize()
  FOR i <- 0 TO 2
    FOR j <- 0 TO 2
      grid[i][j] <- 0
    ENDFOR
  ENDFOR
ENDMETHOD

METHOD Matrix2D::SetElement(row, col, value)
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    grid[row][col] <- value
  ELSE
    DISPLAY "Invalid matrix index"
  ENDIF
ENDMETHOD

METHOD Matrix2D::GetElement(row, col) RETURNS REAL
```

```
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    RETURN grid[row][col]
  ELSE
    DISPLAY "Invalid matrix index"
    RETURN 0
  ENDIF
ENDMETHOD

METHOD Matrix2D::Display()
  FOR i <- 0 TO 2
    FOR j <- 0 TO 2
      DISPLAY grid[i][j], " "
    ENDFOR
    DISPLAY NEWLINE
  ENDFOR
ENDMETHOD

FUNCTION ComputeCrossTrace(A : Matrix2D, B : Matrix2D) RETURNS REAL
  DECLARE sum : REAL <- 0
  FOR i <- 0 TO 2
    sum <- sum + A.grid[i][i] * B.grid[i][i]
  ENDFOR
  RETURN sum
ENDFUNCTION

BEGIN
  DECLARE A, B : Matrix2D
  A.Initialize()
  B.Initialize()
  // Set diagonal elements from the sample
  A.SetElement(0,0,2); A.SetElement(1,1,4); A.SetElement(2,2,6)
  B.SetElement(0,0,1); B.SetElement(1,1,3); B.SetElement(2,2,5)
  DISPLAY "Cross trace: ", ComputeCrossTrace(A, B)
END
```

3. Dry-run trace: Compute $2\times1 + 4\times3 + 6\times5 = 2 + 12 + 30 = 44$.

Index i	A.grid[i][i]	B.grid[i][i]	Product	Running sum
0	2	1	2	2
1	4	3	12	14
2	6	5	30	44

Expected result: Cross trace = 44.

Question 3: Student Academic Roster via Array of Objects

Topic: Arrays of Objects, Member Functions, Sorting Objects.

Scenario: Create StudentRecord objects containing roll number, name, three subject marks, percentage and grade. Use a default constructor, ReadData() and private CalculateResult(). Grade thresholds: A ≥ 85%, B ≥ 70%, C ≥ 50%, otherwise F. Create an array of three objects and sort descending by percentage using Bubble Sort.

Student Tasks: Write StudentRecord and the sorting driver. Show sorting passes and swaps in a trace table.

ANSWER

1. Design overview: Each object stores one student record. CalculateResult() computes the average of the three marks and assigns a grade. Bubble Sort swaps complete object records when a lower percentage appears before a higher one.

2. Structured pseudocode:

```
CLASS StudentRecord
PRIVATE:
  rollNo : INTEGER
  studentName : STRING
  marks : ARRAY[3] OF REAL
  percentage : REAL
  grade : CHARACTER

  METHOD CalculateResult()
    percentage <- (marks[0] + marks[1] + marks[2]) / 3
    IF percentage >= 85 THEN
      grade <- 'A'
    ELSE IF percentage >= 70 THEN
      grade <- 'B'
    ELSE IF percentage >= 50 THEN
      grade <- 'C'
    ELSE
      grade <- 'F'
    ENDIF
  ENDMETHOD

PUBLIC:
  CONSTRUCTOR()
    rollNo <- 0
    studentName <- ""
    marks[0] <- 0
    marks[1] <- 0
    marks[2] <- 0
    percentage <- 0
    grade <- 'F'
  ENDCONSTRUCTOR

  METHOD ReadData(r, name, m1, m2, m3)
```

```
    rollNo <- r
    studentName <- name
    marks[0] <- m1
    marks[1] <- m2
    marks[2] <- m3
    CALL CalculateResult()
  ENDMETHOD

  METHOD GetPercentage() RETURNS REAL
    RETURN percentage
  ENDMETHOD

  METHOD Display()
    DISPLAY rollNo, studentName, percentage, grade
  ENDMETHOD
ENDCLASS

BEGIN
  DECLARE students : ARRAY[3] OF StudentRecord
  students[0].ReadData(101, "Asha", 90, 88, 92)
  students[1].ReadData(102, "Bala", 75, 70, 80)
  students[2].ReadData(103, "Charan", 95, 94, 96)

  FOR i <- 0 TO 1
    FOR j <- 0 TO 1-i
      IF students[j].GetPercentage() <
         students[j+1].GetPercentage() THEN
        SWAP students[j], students[j+1]
      ENDIF
    ENDFOR
  ENDFOR

  DISPLAY "Rank list:"
  FOR i <- 0 TO 2
    students[i].Display()
  ENDFOR
END
```

3. Sample percentages: Asha = 90%; Bala = 75%; Charan = 95%.

Pass	Comparison	Action	Order after pass
Initial	—	—	Asha 90, Bala 75, Charan 95
1	90 < 75? No	No swap	Asha 90, Bala 75, Charan 95
1	75 < 95? Yes	Swap Bala and Charan	Asha 90, Charan 95, Bala 75
2	90 < 95? Yes	Swap Asha and Charan	Charan 95, Asha 90, Bala 75

Expected sorted rank list:

```
103 Charan 95.00 A
101 Asha 90.00 A
102 Bala 75.00 B
```

Question 4: Flexible Logger with Overloaded Functions & Inline Helpers

Topic: Function Overloading, Default Arguments, Inline Functions.

Scenario: Create an industrial logging engine. Inline FormatPrefix(prefixTag) returns a formatted tag. LogEvent(message) logs general information with INFO prefix. The second overload accepts message, severityLevel defaulting to 1, and timestamp defaulting to 00:00:00; severity values 1, 2, 3 map to LOW, MEDIUM, CRITICAL. The third overload accepts errorCode and systemModule and formats a hardware fault message with hexadecimal code.

Student Tasks: Write pseudocode for signatures and inline definitions. Demonstrate calls with one, two and three arguments and trace default argument resolution.

ANSWER

1. Design overview: FormatPrefix is an inline helper. The overloads are distinguished by parameter types/count. Calls with a message use the string-first overload; an integer and module string use the hardware-fault overload.

2. Structured pseudocode:

```
INLINE FUNCTION FormatPrefix(prefixTag : STRING) RETURNS STRING
  RETURN "[" + prefixTag + "] "
ENDFUNCTION

FUNCTION LogEvent(message : STRING)
  DISPLAY FormatPrefix("INFO"), message
ENDFUNCTION

FUNCTION LogEvent(message : STRING,
                  severityLevel : INTEGER = 1,
                  timestamp : STRING = "00:00:00")
  DECLARE severity : STRING
  IF severityLevel = 1 THEN
    severity <- "LOW"
  ELSE IF severityLevel = 2 THEN
    severity <- "MEDIUM"
  ELSE IF severityLevel = 3 THEN
    severity <- "CRITICAL"
  ELSE
    severity <- "UNKNOWN"
```

```
  ENDIF
  DISPLAY "[", severity, "] @ ", timestamp, " : ", message
ENDFUNCTION

FUNCTION LogEvent(errorCode : INTEGER, systemModule : STRING)
  DISPLAY "[HARDWARE_FAULT] Module: ", systemModule,
          " -> Error Code: 0x", HEX(errorCode)
ENDFUNCTION

BEGIN
  LogEvent("System started")
  LogEvent("Temperature high", 2)
  LogEvent("Sensor disconnected", 3, "14:35:10")
  LogEvent(255, "ENGINE")
END
```

3. Dry-run trace: The one-string call selects overload 1. The two-string/integer call selects overload 2 and uses the default timestamp. The three-argument call uses both supplied severity and timestamp. The integer/string call selects overload 3.

Call	Selected overload	Resolved arguments / output
LogEvent("System started")	Overload 1	[INFO] System started
LogEvent("Temperature high", 2)	Overload 2	severity=MEDIUM; timestamp=00:00:00
LogEvent("Sensor disconnected", 3, "14:35:10")	Overload 2	severity=CRITICAL; timestamp=14:35:10
LogEvent(255, "ENGINE")	Overload 3	[HARDWARE_FAULT] Module: ENGINE -> Error Code: 0xFF

Expected output:

```
[INFO] System started
[MEDIUM] @ 00:00:00 : Temperature high
[CRITICAL] @ 14:35:10 : Sensor disconnected
[HARDWARE_FAULT] Module: ENGINE -> Error Code: 0xFF
```

Question 5: Hardware Register Bank with Return-by-Reference

Topic: Return by Reference, Arrays within a Class, Lvalue Semantics.

Scenario: RegisterBank stores eight integer registers and an errorSink. Its constructor initializes registers to zero and errorSink to -1. GetRegister(index) returns a reference to a valid register; for an invalid index it alerts and returns a reference to errorSink. The client writes values through the function call, reads through a reference, and attempts an out-of-bounds assignment.

Student Tasks: Write pseudocode for RegisterBank and its driver. Explain how return-by-reference allows func() = val.

ANSWER

1. Design overview: GetRegister returns an alias to an existing array element, so the returned expression behaves as an lvalue. For invalid indices, it returns the persistent errorSink member rather than an invalid memory location.

2. Structured pseudocode:

```
CLASS RegisterBank
PRIVATE:
  registers : ARRAY[8] OF INTEGER
  errorSink : INTEGER

PUBLIC:
  CONSTRUCTOR()
    FOR i <- 0 TO 7
      registers[i] <- 0
    ENDFOR
    errorSink <- -1
  ENDCONSTRUCTOR

  METHOD GetRegister(index : INTEGER) RETURNS REF INTEGER
    IF index >= 0 AND index < 8 THEN
      RETURN REF registers[index]
    ELSE
      DISPLAY "Alert: Register index out of bounds"
      RETURN REF errorSink
    ENDIF
  ENDMETHOD

  METHOD DisplayState()
    FOR i <- 0 TO 7
      DISPLAY "Register[", i, "] = 0x", HEX(registers[i])
    ENDFOR
  ENDMETHOD
ENDCLASS

BEGIN
  DECLARE bank : RegisterBank
  bank.GetRegister(0) <- 170
  bank.GetRegister(1) <- 255

  DECLARE readVal : INTEGER
  readVal <- bank.GetRegister(0)
  DISPLAY "Read register 0: 0x", HEX(readVal)

  bank.GetRegister(10) <- 999
  DISPLAY "Error sink: ", bank.GetRegister(-1)
  bank.DisplayState()
```

```
END
```

3. Dry-run trace: Register 0 receives 170 (0xAA); register 1 receives 255 (0xFF). Reading register 0 returns 170. Index 10 is invalid, so the function returns errorSink; assigning 999 changes errorSink, not the register array.

Step	Operation	Effect
1	Constructor	All 8 registers = 0; errorSink = -1
2	GetRegister(0) <- 170	registers[0] = 170 (0xAA)
3	GetRegister(1) <- 255	registers[1] = 255 (0xFF)
4	readVal <- GetRegister(0)	readVal = 170 (0xAA)
5	GetRegister(10) <- 999	Alert; errorSink becomes 999

Expected state:

```
Register[0] = 0xAA
Register[1] = 0xFF
Register[2] = 0x0
Register[3] = 0x0
Register[4] = 0x0
Register[5] = 0x0
Register[6] = 0x0
Register[7] = 0x0
Error sink = 999
```

Return-by-reference explanation: A function returning REF INTEGER returns an alias to an existing integer object. Therefore, bank.GetRegister(0) <- 170 assigns directly to registers[0]. The returned reference must refer to an object whose lifetime continues after the function returns; returning a local variable would create a dangling reference.

Important Notes

• The debugging answers include complete corrected programs and expected output.

• Pseudocode follows the source question requirements and uses explicit class/method structure.

• Section B Question 1 does not specify opening balances; the trace explicitly assumes $200,000 for Branch1 and $100,000 for Branch2.

• Section B Question 3 uses sample marks (90,88,92), (75,70,80), and (95,94,96) because the source asks for a trace but does not supply student marks.

• Section B Question 4 demonstrates four calls to cover the one-, two-, and three-argument forms and the hardware-fault overload.